

Systèmes d'exploitation I

Chapitre 3 : Communication interprocessus - Les signaux

Moufida Maimour

TELECOM Nancy – 2ème année

2025-2026

Sommaire

- 1 Introduction
- 2 Communication par signaux : principe et utilité
- 3 Terminaux, sessions et groupe en Unix
- 4 Exemples d'utilisation des signaux
- 5 Conclusion

Communication inter-processus (IPC) dans Unix

→ Aspect central de la programmation système

→ Moyens de communication entre processus sous Unix

- Signaux: suite de cette séance
- Fichiers et tubes (*pipes*, FIFOs): chapitre 4.
- Mémoire partagée et sémaphores: Systèmes II
- Sockets (dans les réseaux, mais aussi en local) : Réseaux Avancés (S8)
- Files de messages

Un signal est un événement **asynchrone** **émis par** l'OS ou un processus **destiné à** un (ou plusieurs) processus

- en shell, `kill <pid>` tue pid

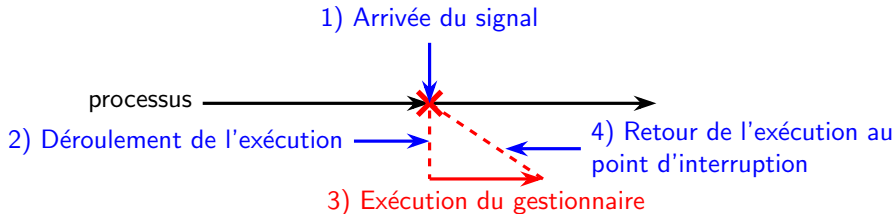
Intérêts et limites

- simplifier le contrôle d'un ensemble de processus (comme le shell)
- pratiques pour traiter des événements liés au temps
- mécanisme de bas niveau **à manipuler avec précaution** (risque de perte de signaux en particulier)

Comparaison avec les interruptions matérielles et exceptions:

- **Analogie** : la réception déclenche l'exécution d'un **gestionnaire** (*handler*)
- **Différences** :
 - ▶ signal reçu par processus
 - ▶ interruption reçue par processeur ;
 - ▶ exception émise par le processeur
- Certains signaux traduisent la réception d'une interruption/exception

Fonctionnement des signaux



Remarques (on va détailler)

- On ne peut évidemment *signaler* que ses propres processus (même uid)
- Différents signaux, identifiés par un nom symbolique (et un entier)
- Gestionnaire par défaut pour chacun
- Gestionnaire vide $\Rightarrow$ ignoré
- On peut changer le gestionnaire (sauf exceptions)
- On peut bloquer un signal : mise en attente, délivré qu'après déblocage
- Limites aux traitements possibles dans le gestionnaire (ex: pas de `signal()`)

Quelques exemples de signaux

Nom symbolique	Cause/signification	Par défaut
SIGINT	frappe du caractère <CTRL-C>	terminaison
SIGTSTP	frappe du caractère <CTRL-Z>	suspension
SIGSTOP	blocage d'un processus (*)	suspension
SIGCONT	continuation d'un processus stoppé	reprise
SIGTERM	demande de terminaison	terminaison
SIGKILL	terminaison immédiate (*)	terminaison
SIGSEGV	erreur de segmentation (violation de protection mémoire)	terminaison + <i>core dump</i>
SIGALRM	top d'horloge (réglée avec alarm)	terminaison
SIGCHLD	terminaison d'un fils	ignoré
SIGUSR1	pas utilisés par le système	terminaison
SIGUSR2	(disponibles pour l'utilisateur)	terminaison

- KILL et STOP: ni bloquables ni ignorables; gestionnaire non modifiable.
 - Valeurs numériques associées (ex: SIGKILL=9), mais pas portable
 - Voir man 7 signal pour d'autres signaux (section 7 du man: conventions)
- core dump:** copie image mémoire sur disque (premières mémoires: toriques → core ; dump=vidanger)

États d'un signal

Signal **pendant** (*pending*)

- Arrivé au destinataire, mais pas encore traité

Signal **traité**

- Le gestionnaire a commencé (et peut-être même fini)

Pendant, mais pas traité? Est-ce possible?

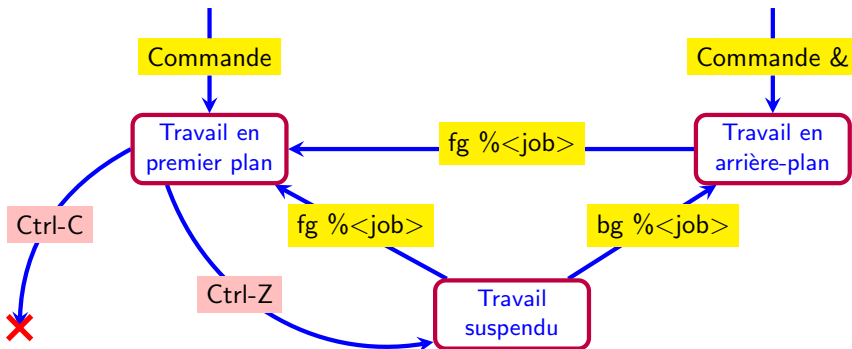
- Il est **bloqué**, càd retardé: il sera délivré lorsque débloqué
- Lors de l'exécution du gestionnaire d'un signal, ce signal est bloqué

Attention: au plus un signal pendant de chaque type

- L'information est codée sur un seul bit
- S'il arrive un autre signal du même type, le second est perdu

États d'un travail

- **Travail** (*job*) = (groupe de) processus lancé par une commande au shell
- Seul le travail en premier plan peut recevoir des signaux du clavier
- Les autres sont manipulés par des commandes



Terminaux, sessions et groupes en Unix

Concept important pour comprendre les signaux sous Unix (détaillé plus tard)

- Une **session** est associée à un **terminal**, donc au login d'un utilisateur par shell
Le processus de ce shell est le **leader de la session**.
- Plusieurs groupes de processus par session. On dit plusieurs **travaux** (*jobs*)
- Au plus un travail interactif (**avant-plan**, *foreground*)
Interagissent avec l'utilisateur via le terminal, seuls à pouvoir lire le terminal
- Plusieurs travaux en **arrière plan** (*background*)
Lancés avec &; Exécution en travail de fond
- Signaux SIGINT (frappe de <CTRL-C>) et SIGTSTP (frappe de <CTRL-Z>) sont passés au groupe interactif et non aux groupes d'arrière-plan

Exemple avec les sessions et groupes Unix

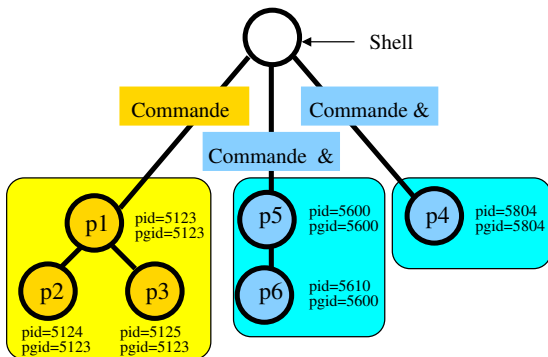
loop.c

```
int main() {  
    printf"processus %d, groupe %d\n", getpid(), getpgrp();  
    while(1) ;  
}
```

```
$ loop & loop & ps  
processus 10468, groupe 10468  
[1] 10468  
processus 10469, groupe 10469  
[2] 10469  
    PID TTY          TIME CMD  
5691 pts/0 00:00:00 bash  
10468 pts/0 00:00:00 loop  
10469 pts/0 00:00:00 loop  
10470 pts/0 00:00:00 ps  
$ fg %1  
loop  
[frappe de control-Z]  
Suspended  
$ jobs  
[1] + Suspended          loop  
[2] - Running             loop
```

```
$ bg %1  
[1] loop &  
$ fg %2  
loop  
[frappe de control-C]  
$ ps  
    PID TTY          TIME CMD  
5691 pts/0 00:00:00 bash  
10468 pts/0 00:02:53 loop  
10474 pts/0 00:00:00 ps  
$ [frappe de control-C]  
$ ps  
    PID TTY          TIME CMD  
5691 pts/0 00:00:00 bash  
10468 pts/0 00:02:57 loop  
10475 pts/0 00:00:00 ps  
$
```

Exemple de travaux



- Signaux CTRL-C et CTRL-Z adressés à **tous** les processus du groupe jaune
- Commandes shell fg, bg et stop pour travaux bleus

Envoyer un signal à un autre processus

Interfaces

- Langage de commande:

```
kill -NOM victime
```

- Appel système:

```
#include <signal.h>

int kill(pid_t victime, int sig);
```

Sémantique: à qui est envoyé le signal?

- Si $victime > 0$, au processus tel que $pid = victime$
- Si $victime = 0$, à tous les processus du même groupe (pgid) que l'émetteur
- Si $victime = -1$:
 - ▶ Si super-utilisateur, à tous les processus sauf système et émetteur
 - ▶ Si non, à tous les processus dont l'utilisateur est propriétaire
- Si $victime < -1$, aux processus tels que $pgid = |victime|$ (tout le groupe)

Redéfinir le gestionnaire associé à un signal (POSIX)

Structure à utiliser pour décrire un gestionnaire

```
struct sigaction {  
    void (*sa_handler)(int);                /* gestionnaire, interface simple */  
    void (* sa_sigaction) (int, siginfo_t *, void *); /* gestionnaire, interface complète */  
    sigset_t sa_mask;                       /* signaux à bloquer pendant le traitement */  
    int sa_flags;                           /* options */  
};
```

- **Gestionnaires particuliers:** SIG_DFL: action par défaut; SIG_IGN: ignorer signal
- Deux types de gestionnaires
 - ▶ sa_handler() connaît le numéro du signal
 - ▶ sa_sigaction() a plus d'infos

Primitive à utiliser pour installer un nouveau gestionnaire

```
#include <signal.h>  
int sigaction(int sig, const struct sigaction *newaction, struct sigaction *oldaction);
```

- Voir man sigaction pour les détails

Autre interface existante : ANSI C

- Peut-être un peu plus simple, mais bien moins puissante et pas thread-safe

Exemple 1 : traitement d'une interruption du clavier

Par défaut, Ctrl-C tue le processus; pour survivre: **il suffit de redéfinir le gestionnaire de SIGINT**

test-int.c

```
#include <signal.h>
void handler(int sig) {                /* nouveau gestionnaire */
    printf("signal SIGINT reçu !\n");
    exit(0);
}
int main() {
    struct sigaction nvt,old;
    memset(&nvt, 0, sizeof(nvt));
    nvt.sa_handler = handler;
    sigaction(SIGINT, &nvt, &old);    /*installe le gestionnaire*/
    pause ();                        /* se bloquer en attente d'un signal */
    printf("Ceci ne sera jamais affiché \n");
}
```

```
$ ./test-int
[frappe de CTRL-C]
signal SIGINT reçu !
$
```

Exercice : modifier ce programme pour qu'il continue après un CTRL-C

Note: il doit rester interruptible, *i.e.* on doit pouvoir le tuer d'un CTRL-C de plus

Exemple 1 : traitement d'une interruption du clavier

Par défaut, Ctrl-C tue le processus; pour survivre: **il suffit de redéfinir le gestionnaire de SIGINT**

test-int.c

```
#include <signal.h>
void handler(int sig) {                               /* nouveau gestionnaire */
    printf("signal SIGINT reçu !\n");
    exit(0);
}
int main() {
    struct sigaction nvt,old;
    memset(&nvt, 0, sizeof(nvt));
    nvt.sa_handler = handler;
    sigaction(SIGINT, &nvt, &old);    /*installe le gestionnaire*/
    pause ();                        /* se bloquer en attente d'un signal */
    printf("Ceci ne sera jamais affiché \n");
}
```

```
$ ./test-int
[frappe de CTRL-C]
signal SIGINT reçu !
$
```

Exercice : modifier ce programme pour qu'il continue après un CTRL-C

Note: il doit rester interruptible, *i.e.* on doit pouvoir le tuer d'un CTRL-C de plus

```
#include <signal.h>
void handler(int sig) {
    printf("signal SIGINT reçu !\n");
}
int main() {
    struct sigaction nvt,old;
    nvt.sa_handler = handler; sigaction(SIGINT, &nvt, &old);
    pause ();
    nvt.sa_handler = SIG_DFL; sigaction(SIGINT, &nvt, &old);
    while (1); }
```

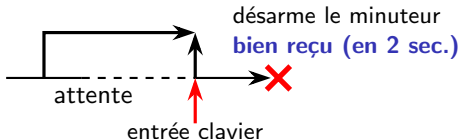
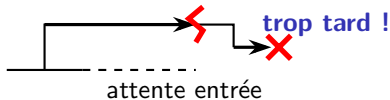
Exemple 2 : temporisation

`unsigned int alarm(unsigned int nb_sec)`

- `nb_sec > 0`: demande l'envoi de SIGALRM après environ `nb_sec` secondes
- `nb_sec = 0`: annulation de toute demande précédente
- Retour: nombre de secondes restantes sur l'alarme précédente

```
#include <signal.h>
void handler(int sig) {
    printf("trop tard !\n");
    exit(1);
}
int main() {
    struct sigaction nvt,old;
    int reponse,restant;
    memset(&nvt, 0, sizeof(nvt));
    nvt.sa_handler = handler;
    sigaction(SIGALRM, &nvt, &old);
```

```
    printf("Entrez un nombre avant 5 sec:");
    alarm(5);
    scanf("%d", &reponse);
    restant = alarm(0);
    printf("bien reçu (en %d secondes)\n",
        5 - restant);
    exit (0);
}
```



Exemple 3 : synchronisation père-fils

→ Fin ou suspension/reprise (changement d'état d'un fils) d'un processus ⇒ SIGCHLD automatique à son père

- **Traitement par défaut:** ignorer ce signal
- **Application:** `wait()` pour éviter les zombies et libérer les ressources système. C'est ce que fait le processus `init` (`pid = 1`) ou `systemd`

```
#include <signal.h>
void handler(int sig) {                               /* nouveau gestionnaire */
    pid_t pid;
    int statut;
    pid = waitpid(-1, &statut, 0); /* attend un fils quelconque */
    return;
}
int main() {
    struct sigaction nvt,old;
    memset(&nvt, 0, sizeof(nvt));
    nvt.sa_handler = handler;
    sigaction(SIGCHLD, &nvt, &old); /* installe le gestionnaire */
    ... <création d'un certain nombre de fils> ...
    exit (0);
}
```

Résumé de la fin du chapitre

- Quelques définitions
 - ▶ pgid: **groupe de processus**; un par processus lancé par shell (et tous ses fils)
 - ▶ Travail d'avant-plan, d'arrière-plan: **lancé avec ou sans &**
- Communication par signaux
 - ▶ Envoyer un signal *<NOM>* à *<victime>*:
 - ★ Langage commande : **kill -NOM victime**
 - ★ API: **kill(pid_t victime, int sig)**
 - victime* > 0: **numéro du pid visé**
 - victime* = 0: **tous les process de ce groupe**
 - victime* = -1: **tous les process accessibles**
 - victime* < -1: **tous les process du groupe *abs(victime)***
 - ★ Astuce: envoyer le signal 9 à tous les processus: **kill -9 -1**
 - ▶ Changer le gestionnaire d'un signal: **sigaction (en POSIX)**
 - ▶ Exemples de signaux
 - ★ SIGINT, SIGSTOP: **Interaction avec le travail de premier plan (ctrl-c, ctrl-z)**
 - ★ SIGTERM, SIGKILL: **demande de terminaison, fin brutale**
 - ★ SIGALARM: **temporisation**
 - ★ SIGCHLD: **relations père-fils**